

Wallace Tree Multiplier

This project and its implementation are based on the concepts and architecture described in the article: Design of Wallace tree multiplier circuit using high performance and low power full adder[1].

1- Introduction

The world is constantly changing, and new technologies are evolving every moment. With the continuous advancement and transformation of the world, numerous developments have also taken place in the field of digital signal processing. It can be observed that signal processing is advancing to higher levels through new inventions. Here, the role of the multiplier comes into play.

The multiplier plays a crucial role in digital signal processing and embedded system applications. In digital signal processing[1], the multiplier is the core component of the processor. The processor functions as the brain of the system. The multiplier is a tool used to amplify (scale) various signals according to specific requirements.

2- Array Multiplier

The multiplication process in an array multiplier consists of three main stages: partial product generation, partial product reduction, and finally, the final addition. The key feature of this method is the parallel computation of all partial products, which provides high speed.

For example, to multiply $1110_2 = 14_{10}$ by $1011_2 = 11_{10}$, we proceed as follows (considering one as the multiplier (1011_2) and the other as the multiplicand (1110_2)):

Bit Index	Bit Value	Left-Shifted Value (by bit index)	Left-Shifted Value (by bit index)
0	1	1110	$1 \times 1110 = 1110$
1	1	11100	$1 \times 11100 = 11100$
2	0	111000	$0 \times 111000 = 000000$
3	1	1110000	$1 \times 1110000 = 1110000$

Final Sum: $1110 + 11100 + 000000 + 1110000 = 10011010_2 = 154_{10}$

To save hardware resources, the partial products can be summed serially. In this method, the multiplicand is added to an accumulator. While the multiplicand is shifted left (to align with the next bit position), the multiplier bits are shifted right. Whenever the least significant bit (LSB) of the multiplier is 1, the

multiplicand is added to the accumulator. This process continues until all bits of the multiplier become zero.

Let's take 1011_2 as the multiplier and 1110_2 as the multiplicand:

Multiplier LSB	Accumulator (Added if multiplier LSB = 1)	Multiplier Right-Shift	Multiplicand Left-Shift
1(1011)	$0000+1110 = 1110$	0101	11100
1(0101)	$1110+11100 = 101010$	0010	111000
0(0010)	-	0001	1110000
1(0001)	$101010+1110000 = 10011010$	0000	11100000
0(0000)	$10011010_2 = 154_{10}$	-	-

3- Wallace Tree Multiplier

The Wallace Tree multiplier is a multiplication algorithm with three main stages. In the first stage, AND gates are used to multiply each bit of one number by each bit of the other number. The result of this operation is a set of partial products.

In the second stage, these partial products are reduced to two numbers. This is done by arranging wires of equal weight (same bit position) in a layer. If there are three or more wires of the same weight, they are fed into a Full Adder. If there are exactly two wires of the same weight, they are fed into a Half Adder. If only one wire remains, it is passed directly to the next layer.

In the third and final stage, the two resulting numbers from the second stage are fed into a carry-propagate adder (e.g., a ripple carry adder or carry lookahead adder) to compute the final product[2].

The partial products are generated in the same way as in previous algorithms (using AND gates). However, instead of simple parallel addition or sequential addition, they are reduced hierarchically using a tree structure consisting of Half Adders and Full Adders.

Let's take 1011_2 as the multiplier and 1110_2 as the multiplicand.

First generate Partial Products and add to the table.

Partial Product Bits						
Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	1	1	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
1	1	1	0	0	0	0
Column 6	Column 5	Column 4	Column 3	Column 2	Column 1	Column 0

Operations to Perform:

Columns	Existing Bits (Only 1s)	Operation	sum	Carry out
Column 0	-	Direct Pass	0	0
Column 1	1→1	Direct Pass	1	0
Column 2	1,1→2	halfadder	0	1
Column 3	1,1,1(Carry 2)→3	fulladder	1	1
Column 4	1,1,1(Carry 3)→3	fulladder	1	1
Column 5	1,1(Carry 4)→2	halfadder	0	1
Column 6	1,1(Carry 5)→2	halfadder	0	1
Column 7	1 (Carry 6)→1	Direct Pass	1	0

sum:

Sum = 100110102 = 15410

4- implementation:

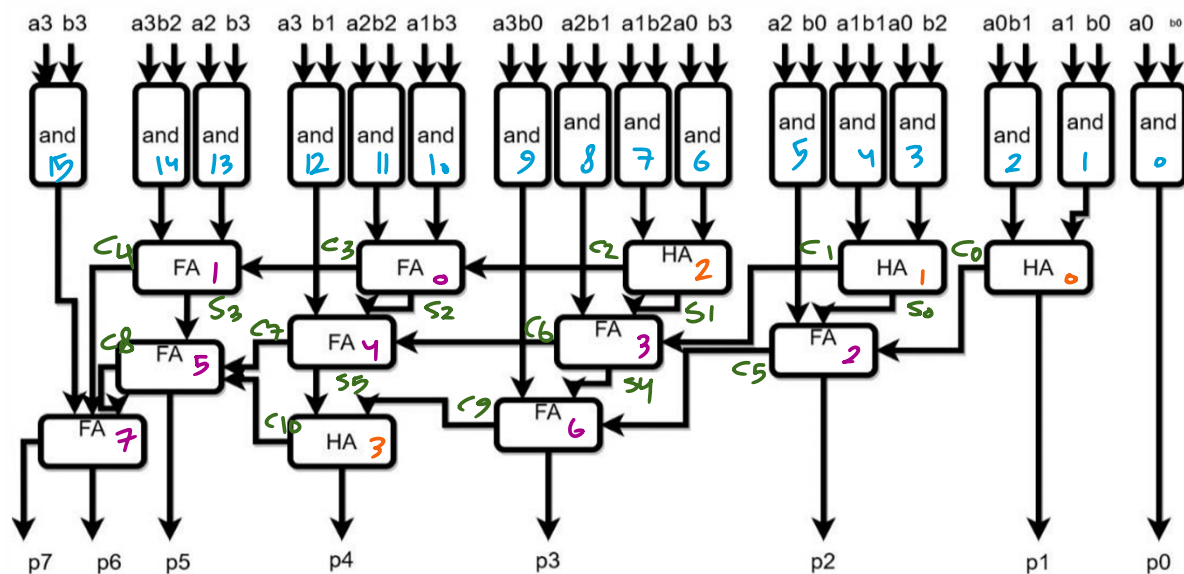


Figure 1 Wallace Tree circuit

According to the figure, 16 AND gates are required to generate the partial products. Then, if there is only one output, it is passed directly; if there are two outputs, they are fed to a half adder; and if there are three outputs, they are fed to a full adder.

4-1- Half Adder Implementation

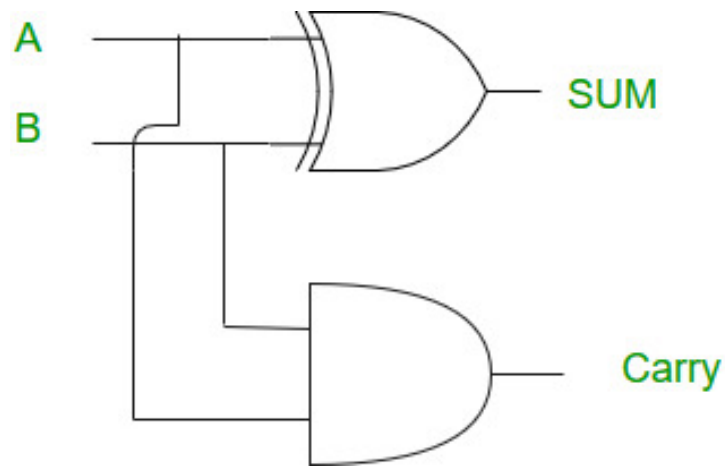


Figure 2 Half Adder circuit [3]

To implement a half adder, we need one XOR and one AND gate.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity HA is
5      port(
6          a,b : in std_logic;
7          sum : out std_logic;
8          cout : out std_logic
9      );
10 end HA;
11
12 architecture half_adder of HA is
13 begin
14     sum <= a xor b;
15     cout <= a and b;
16 end half_adder;
--

```

Figure 3 Half Adder VHDL code

4-2- Full Adder Implementation

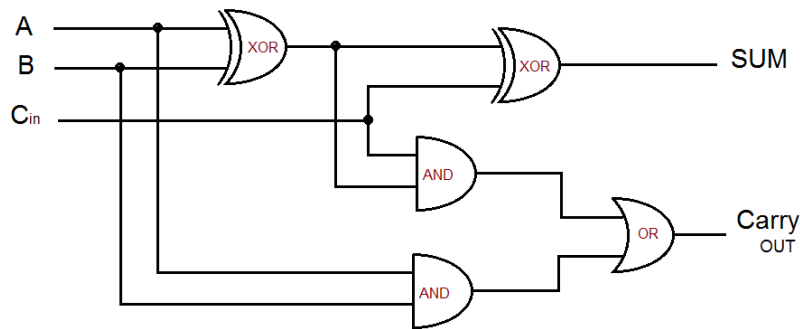


Figure 4 Full Adder circuit [4]

To implement a full adder, we need two XOR gates, two AND gates, and one OR gate.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity FA is
5      port(
6          a,b,cin : in std_logic;
7          sum : out std_logic;
8          cout : out std_logic
9      );
10 end FA;
11
12 architecture full_adder of FA is
13 begin
14     sum <= a xor b xor cin;
15     cout <= (a and b) or (b and cin) or (a and cin);
16 end full_adder;
17

```

Figure 5 Full Adder VHDL code

4-3- Wallace Tree Implementation

Inputs: Two 4-bit vectors.

Output: An 8-bit vector representing the product.

First, all partial products are generated using AND gates. These partial products are then reduced over several stages using Half Adders and Full Adders. By compressing these results in parallel, the number of addition stages is reduced, thereby increasing the multiplication speed. Finally, the output bits are assigned, which represent the final product.

```

library ieee;
use ieee.std_logic_1164.all;

entity wallace is
port(
    A,B : in  std_logic_vector(3 downto 0);
    p : out std_logic_vector(7 downto 0)
);
end wallace;

architecture design of wallace is
    type andout  is array (0 to 15) of std_logic;
    type sum_array  is array (0 to 6) of std_logic;
    type carry_array is array (0 to 10) of std_logic;

    signal and_out : andout;
    signal s : sum_array;
    signal c : carry_array;
begin
    and_out(0) <= A(0) and B(0);
    and_out(1) <= A(1) and B(0);
    and_out(2) <= A(0) and B(1);
    and_out(3) <= A(0) and B(2);
    and_out(4) <= A(1) and B(1);
    and_out(5) <= A(2) and B(0);
    and_out(6) <= A(0) and B(3);
    and_out(7) <= A(1) and B(2);
    and_out(8) <= A(2) and B(1);
    and_out(9) <= A(3) and B(0);
    and_out(10) <= A(1) and B(3);
    and_out(11) <= A(2) and B(2);
    and_out(12) <= A(3) and B(1);
    and_out(13) <= A(2) and B(3);
    and_out(14) <= A(3) and B(2);
    and_out(15) <= A(3) and B(3);

    -- 1st row
    HA0: entity work.HA port map(a => and_out(1),b => and_out(2),sum => p(1),cout => c(0));
    HA1: entity work.HA port map(a => and_out(3),b => and_out(4),sum => s(0),cout => c(1));
    HA2: entity work.HA port map(a => and_out(6),b => and_out(7),sum => s(1),cout => c(2));
    FA0: entity work.FA port map(a => and_out(10),b => and_out(11),cin => c(2),sum => s(2),cout => c(3));
    FA1: entity work.FA port map(a => and_out(13),b => and_out(14),cin => c(3),sum => s(3),cout => c(4));

    -- 2nd row
    FA2: entity work.FA port map(a => s(0),b => and_out(5),cin => c(0),sum => p(2),cout => c(5));
    FA3: entity work.FA port map(a => s(1),b => and_out(8),cin => c(1),sum => s(4),cout => c(6));
    FA4: entity work.FA port map(a => s(2),b => and_out(12),cin => c(6),sum => s(5),cout => c(7));
    FA5: entity work.FA port map(a => s(3),b => c(7),cin => c(10),sum => p(5),cout => c(8));

    -- 3rd row
    FA6: entity work.FA port map(a => s(4),b => and_out(9),cin => c(5),sum => p(3),cout => c(9));
    HA3: entity work.HA port map(a => c(9),b => s(5),sum => p(4),cout => c(10));
    FA7: entity work.FA port map(a => c(8),b => and_out(15),cin => c(4),sum => p(6),cout => p(7));

    p(0) <= and_out(0);
end design;

```

Figure 6 Wallace Tree Multiplier VHDL code

4-4- Waveform

Testbench Implementation:

```

1  |library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity tb_wallace is
5  end tb_wallace;
6
7  architecture behavior of tb_wallace is
8      component wallace
9          port(
10             A,B : in std_logic_vector(3 downto 0);
11             p : out std_logic_vector(7 downto 0)
12          );
13      end component;
14
15      signal A_tb : std_logic_vector(3 downto 0) := (others => '0');
16      signal B_tb : std_logic_vector(3 downto 0) := (others => '0');
17      signal P_tb : std_logic_vector(7 downto 0);
18
19  begin
20      w: wallace
21          port map (
22             A => A_tb,
23             B => B_tb,
24             p => P_tb
25          );
26
27      proc: process
28      begin
29          A_tb <= "1110"; B_tb <= "1011";
30          wait for 10 ns;
31
32          A_tb <= "0111"; B_tb <= "1001";
33          wait for 10 ns;
34
35          A_tb <= "1111"; B_tb <= "1111";
36          wait for 10 ns;
37          wait;
38      end process;

```

Figure 7 Wallace Tree Testbench VHDL code

Output:



Figure 8 Simulation Waveform

5- References

- [1] K. B. D. Kumar, L. B. Reddy, V. Pudi, and S. Bodapati, "Design of low area and low power systolic serial parallel multiplier using CNTFETS," in 2021 IEEE International Symposium on Smart Electronic Systems (iSES), 2021, pp. 139–142.
- [2] F.-Y. Gu, C. Lin, and J.-W. Lin, "A low-power and high-accuracy approximate multiplier with reconfigurable truncation," IEEE Access, vol. 10, pp. 60447–60458, 2022.
- [3] GeeksforGeeks, "Half Adder," GeeksforGeeks, [Online]. Available: https://media.geeksforgeeks.org/wp-content/cdn-uploads/Half_Adder.jpg.
- [4] TheoryCircuit, "Full Adder Circuit," TheoryCircuit, [Online]. Available: <https://theorycircuit.com/wp-content/uploads/2018/07/full-adder-circuit.png>.